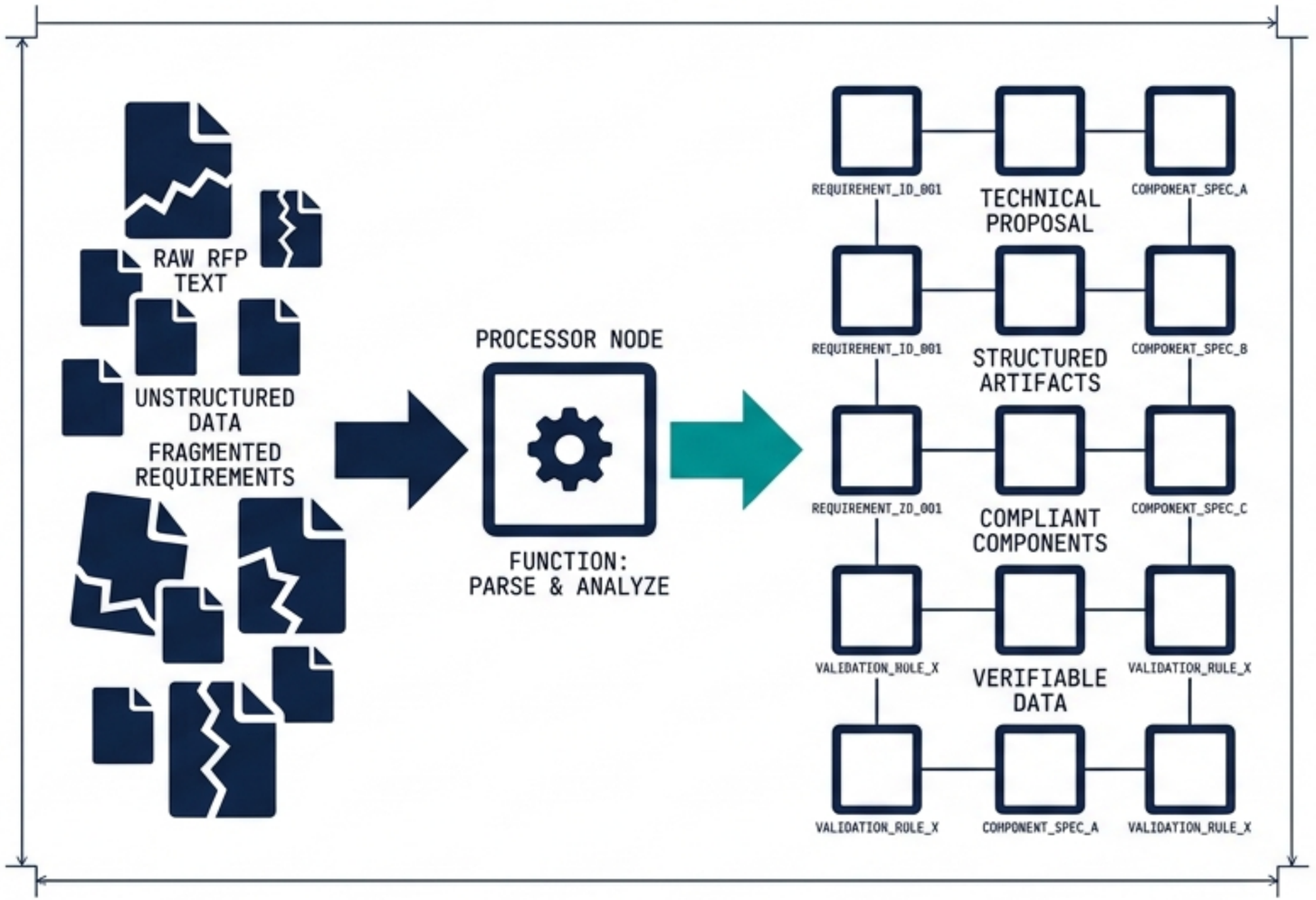
RFP Copilot Orchestration Prototype

LangGraph Architecture & Neuro-Symbolic Validation

Dominique Loyer | Forward Deployed Engineering Demo

Gemini Notebook

Transforming Unstructured Requirements into Verifiable Artifacts



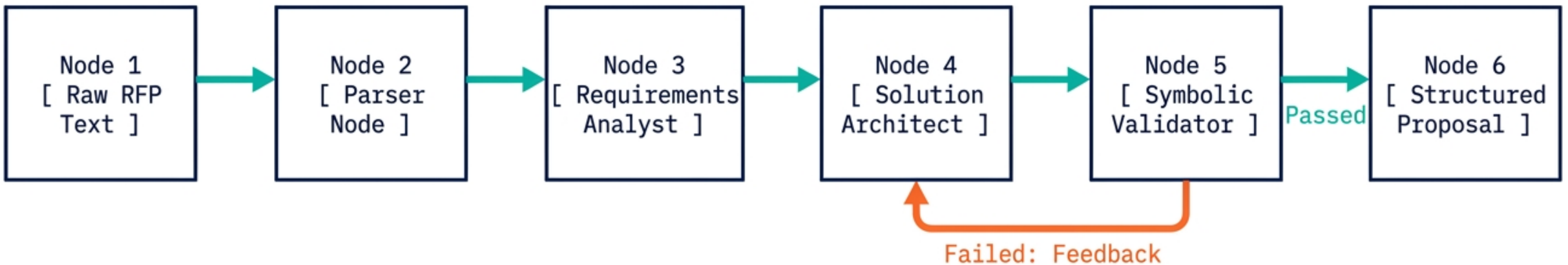
GOAL:

Automate the generation of technical proposals by parsing raw RFP requirements into highly structured, compliant artifacts.

CORE INNOVATION:

Utilizing LangGraph to enforce a strict architectural separation between cognitive tasks (parsing, analysis, drafting) and a symbolic validation layer.

Gemini Notebook



System Note

Prototype demonstration: Parsing and proposal generation are currently stubbed. Symbolic checks validate declared fields, not legal compliance.

Planned Integrations

PDF Ingestion | LLM Generation | Retrieval (RAG) | Telemetry | Evaluation Harness

Gemini Notebook

LangGraph Orchestration & Mutating Shared State

A graph structure is intentionally chosen to strictly separate distinct cognitive steps (parsing vs. analysis vs. drafting).

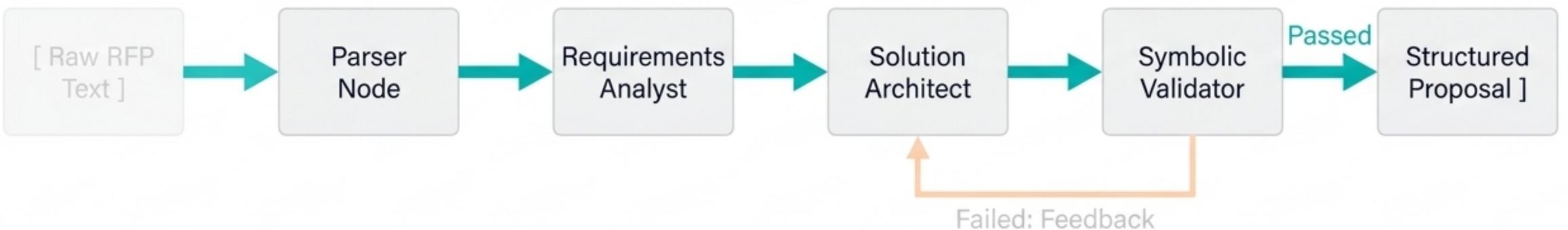
The shared state carries this payload. As the workflow traverses the graph, individual nodes mutate this state, appending extracted requirements, drafting solutions, and injecting validation feedback.



Gemini Notebook

Forward Execution: The Unimpeded Workflow

- The workflow moves linearly through the Parser, Requirements Analyst, and Solution Architect before reaching the Symbolic Validator.
- Prototype State: Parsing and proposal generation are explicitly stubbed in this phase to allow clear, isolated inspection of the graph structure and routing logic.
- Execution Example: In the successful path, the drafted solution already contains required markers (e.g., a 'GDPR marker'). The validator detects this, updating the validation_status to true, triggering a seamless Teal progression to the Structured Proposal.



Gemini Notebook

sysCRED-Inspired Neuro-Symbolic Validation

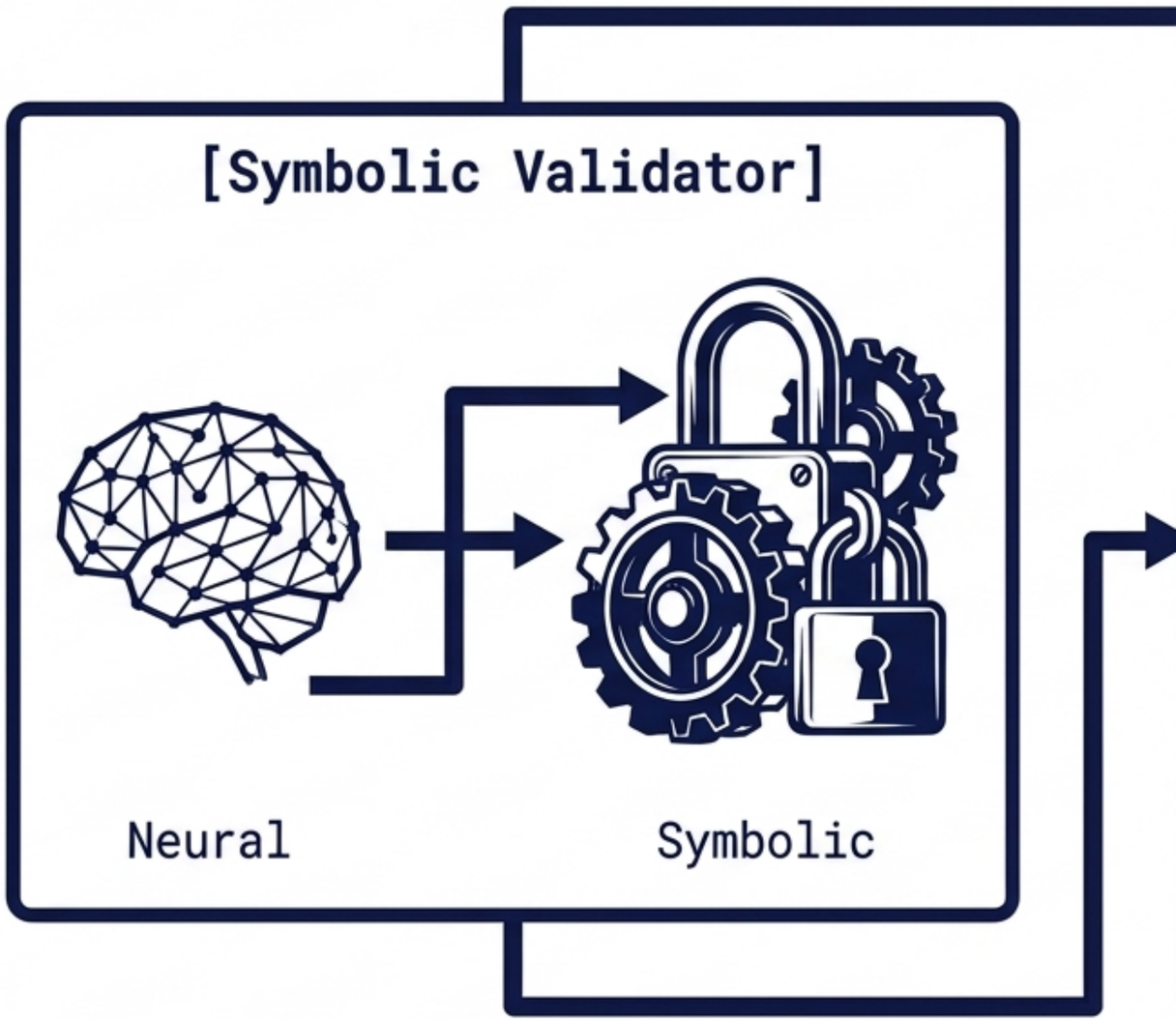
Core Philosophy: Distinct separation between what a generative language model proposes and what explicit, deterministic rules can actually check.

Prototype Scope

Currently implemented as a deliberately narrow string check confirming the presence of declared fields.

Constraint Clarification

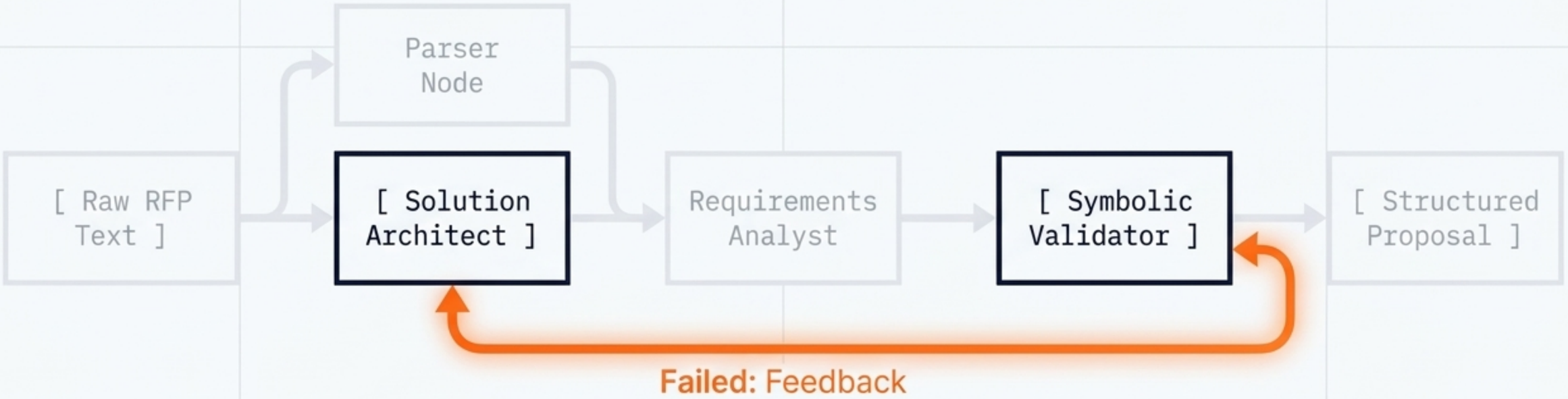
This logic confirms field declaration; it does not establish legal compliance or guarantee the architecture satisfies every requirement.



Gemini Notebook

The Refinement Loop: Conditional Routing via Feedback

- Trigger: The `should_loop_back` routing condition is activated when the Symbolic Validator fails a check.
- Mechanism: The loop routes the system state back to the `[Solution Architect]`.
- Requirement: The `Architect` node is programmed to consume the newly updated feedback string within the shared state. This ensures that the next generative attempt incorporates the validator's corrections to make a meaningful architectural change.



Gemini Notebook

Architectural Limits: Enforcing Exhaustion Constraints

The Flaw

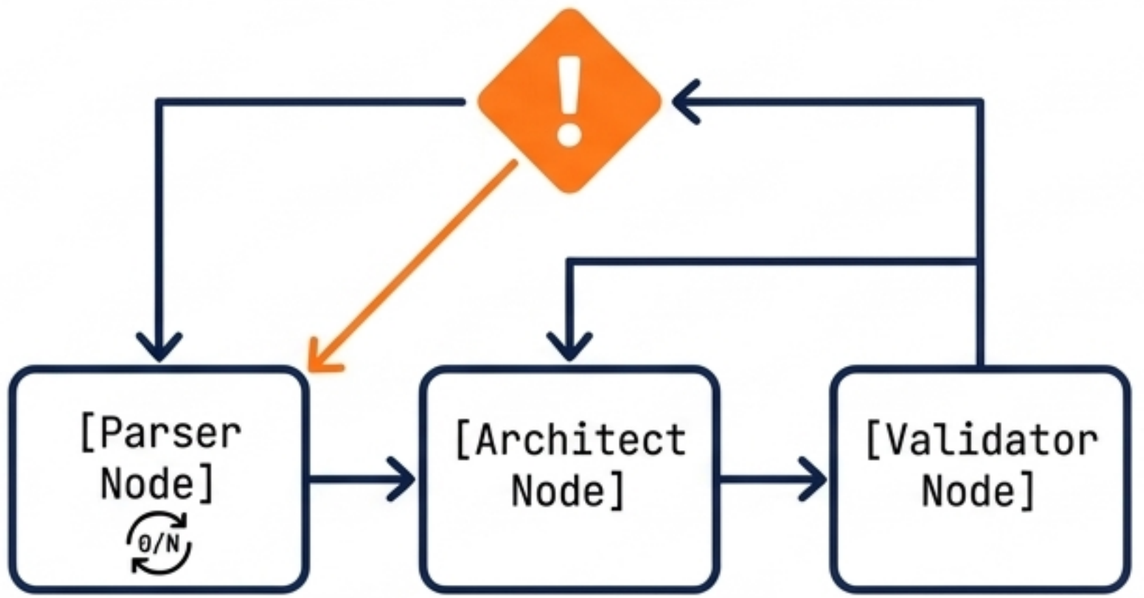
In the current iteration, the `iteration_count` is incremented in the [Parser Node]. However, the refinement loop bypasses the parser, returning directly to the [Solution Architect]. Therefore, the intended retry limit is not actively enforced.

The Correction

An attempt counter must be incremented within the refinement cycle to properly track budgets.

End State Logic

Upon budget exhaustion, the system must forcefully terminate, and the output must be explicitly flagged as unvalidated to prevent an unverified draft from being presented as an approved proposal.



Gemini Notebook

Component Maturity: Prototype vs. Production Target

	Current Prototype	Production Target
Validation	Narrow string check for declared fields.	**Typed fields, explicit requirement-to-evidence mappings, and discrete predicates for each constraint.**
Generation	Stubbed proposal data for routing inspection.	**Live LLM solution drafting and generation.**
Ingestion	Raw text input.	**Multi-modal PDF ingestion and Retrieval-Augmented Generation (RAG).**

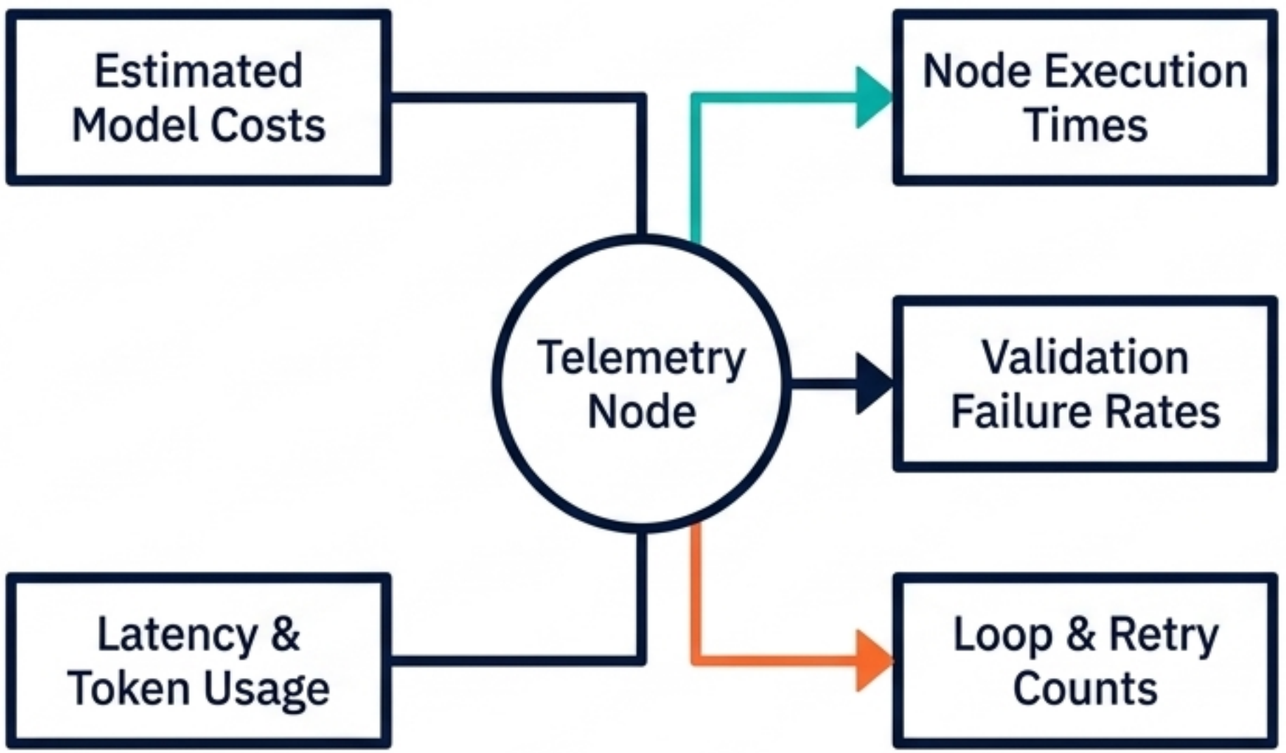
Gemini Notebook

Observability & The Evaluation Harness

Next Milestone: Implementation of `tests/test_eval_harness.py` and structural tracking in `DECISIONS.md`.. based on the planned observability metrics.

Required Test Coverage

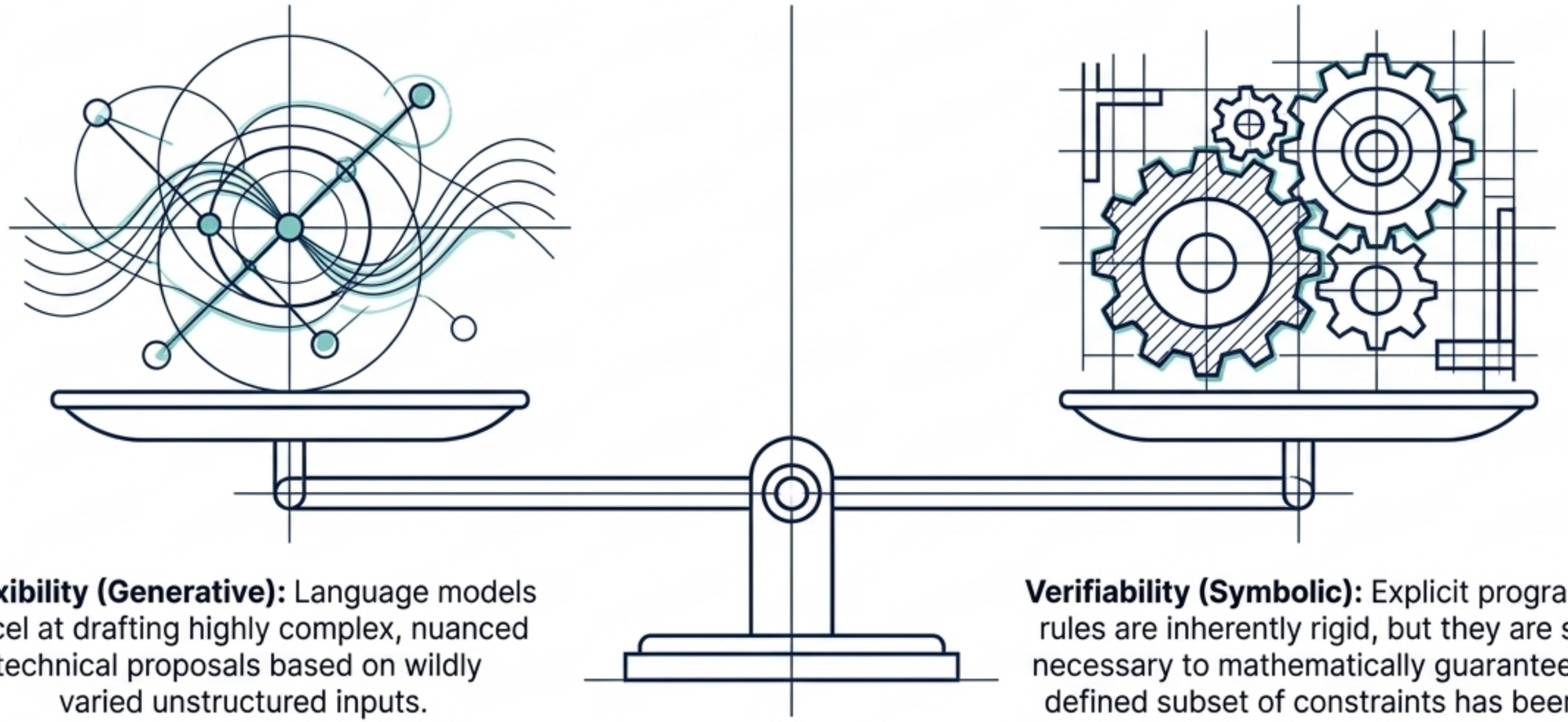
- [x] Successful validation path
- [x] Missing field detection
- [x] Unsuccessful refinement (Feedback capture)
- [x] Retry exhaustion termination



Gemini Notebook

Core Synthesis: Flexibility vs. Verifiability

The central design trade-off defining the RFP Copilot architecture.
An enterprise architecture must separate these concerns.



Flexibility (Generative): Language models excel at drafting highly complex, nuanced technical proposals based on wildly varied unstructured inputs.

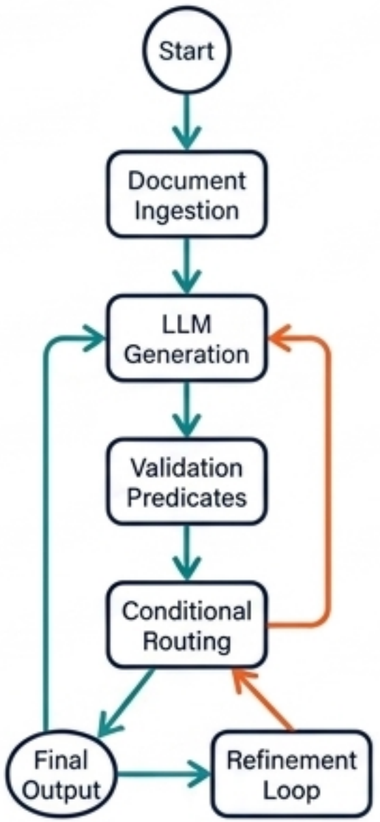
Verifiability (Symbolic): Explicit programmatic rules are inherently rigid, but they are strictly necessary to mathematically guarantee that a defined subset of constraints has been met.

Gemini Notebook

System Readiness & Remaining Engineering

Established Success

The prototype successfully proves the LangGraph workflow structure, shared state payload mutation, and conditional routing logic.



Remaining Work

- Connect real document ingestion and LLM generation.
- Strengthen validation predicates from strings to types.
- Implement reproducible tests with full observability.

Thank you for your time.

Gemini Notebook